Course: Enterprise Real-Time Data Engineering

Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)

Course Portal: fakhr.me/KafkaSpark

Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

S00: Hands-on Refresher Exercises & Practice Lab

Technical Warm-up: Python, Linux CLI, Docker, SQL & Git

Audience: Trainees

Duration: 75 Minutes (Sunday 09:00 – 10:15)

Exercise 1: Python Virtual Environment & Basic Script

Run the following commands in your Linux/WSL terminal:

```
# 1. Create a clean practice folder
mkdir -p ~/kafka_warmup && cd ~/kafka_warmup

# 2. Initialize an isolated virtual environment
python3 -m venv .venv
source .venv/bin/activate

# 3. Create a test script
cat << 'EOF' > test_env.py
import sys
import json
import datetime as dt

print(f"Python Runtime: {sys.version.split()[0]}")
event = {"order_id": 991, "status": "CONFIRMED", "timestamp": int(dt.datetime.now(dt.timezon
e.utc).timestamp() * 1000)}
print(f"Sample JSON Event: {json.dumps(event)}")
EOF

# 4. Run the script
python test_env.py
```

Expected Output:

```
Python Runtime: 3.10.x (or higher)
Sample JSON Event: {"order_id": 991, "status": "CONFIRMED", "timestamp": 1730819200000}
```

Exercise 2: Terminal Navigation & Log Inspection

Practice filtering output and checking status codes:

```
# 1. Create a dummy server log
cat << 'EOF' > app.log
2026-11-05 09:15:01 INFO Connection established with broker 1
2026-11-05 09:15:04 WARN Consumer lag exceeded 500 records
2026-11-05 09:15:10 ERROR Broker 2 heartbeat timeout: Connection refused
2026-11-05 09:15:15 INFO Rebalance completed successfully
EOF

# 2. Filter for error lines only
grep "ERROR" app.log

# 3. Check exit code of the grep command
echo $?
```

Expected Output:

```
2026-11-05 09:15:10 ERROR Broker 2 heartbeat timeout: Connection refused
0
```

(Notice: `echo $?` outputs `0` because `grep` found a matching line!)

Exercise 3: Docker Container Inspection

Verify your workstation's Docker daemon and running cluster containers:

```
# Check running containers
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Expected Output:

You should see `kafka1`, `kafka2`, `kafka3`, `schema-registry`, `postgres`, `connect`, `ksqldb-server`, and `kafka-ui` listed.

Exercise 4: Hands-On SQL Queries in Python (Zero Installation)

Python includes SQLite in its standard library. Run this one-liner script to test your SQL logic:

```
python -c "
import sqlite3
con = sqlite3.connect(':memory:')
con.executescript('''
CREATE TABLE orders (order_id INT, customer TEXT, amount REAL);
INSERT INTO orders VALUES (1, 'Nile-01', 1200.0), (2, 'Nile-02', 350.0), (3, 'Nile-01', 800.
0);
''')
cur = con.cursor()
print('--- Customer Total Spend ---')
for row in cur.execute('SELECT customer, SUM(amount), COUNT(*) FROM orders GROUP BY customer
ORDER BY customer'):
    print(f'Customer: {row[0]} | Total Spent: {row[1]} EGP | Orders: {row[2]}')
"
```

Expected Output:

```
--- Customer Total Spend ---
Customer: Nile-01 | Total Spent: 2000.0 EGP | Orders: 2
Customer: Nile-02 | Total Spent: 350.0 EGP | Orders: 1
```

Exercise 5: Git & Java Verification

Confirm your version control and Java installation:

```
# Verify Git
git --version

# Verify Java (JDK 17 or 21 required for Apache Spark)
java -version
```

If all 5 exercises run successfully, your workstation is **100% certified and ready for Session S01**!

Enterprise Real-Time Data Engineering · Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me) · Course Materials: fakhr.me/KafkaSpark · Repository: github.com/fakhruldeen/KafkaSpark

---

🌟 Gold Stretch Challenge: Container Networking & Subnet Inspection

Distinguished Engineer Challenge: If you complete the core exercises before time expires, execute this advanced infrastructure verification:

1. Inspect the internal Docker network created for the multi-broker cluster:

```bash
docker network inspect kafka-spark-lab-v8_default 2>/dev/null || docker network inspect $(docker network ls -q --filter name=default)
```

2. Verify the IP addresses and subnet masks assigned to `kafka1`, `kafka2`, and `kafka3`.
3. Test direct inter-container socket reachability from inside the Spark master container:

```bash
docker exec -it spark-master nc -zv kafka1 9092
```

4. Verify that the DNS resolver inside Docker resolves container hostnames with sub-millisecond round-trip times.